

LLM Training with LoRA — Continued Pre-training and SFT

Part 2 – Submission due 10.06.2026

In the following two assignments you will practice training a language model in two stages — continued pre-training and instruction fine-tuning. In both cases you will need to evaluate the models and write conclusions.

In Part 1 you adapted a pre-trained language model the Sherlock Holmes corpus via continued pre-training (next-token prediction with LoRA). In Part 2 (This task) you will take that adapted model and teach it to **answer question** grounded in the same corpus through Supervised Fine-Tuning (SFT)

Keep in mind: this assignment builds directly on your Part 1 checkpoint. If for some reason you don't have access to your previous model (or you don't have it) please write it down, but keep in mind that the purpose is to use that model exactly.

General Instructions:

- You are encouraged to use every tool at your disposal, including AI coding assistants (Claude, Copilot, ChatGPT - please add a paragraph to your report describing how you used the AI assistant in this task.
- We have allocated a special queue in the department lab for this course. To use it, please use the following settings:

`qos course, --part rtx2080, --gpu 1, --account cours_ibm26`

. This GPU should be enough for the tasks, but you are free to use larger GPUs or Colab.
- The submission must include both a report and the code you used. You are free to use notebooks or Python scripts. The report (PDF file) must include:
 - Everything explicitly requested in the assignment.
 - Which GPUs did you use.
 - All assumptions you took.
 - [Important] What did not work: log your mistakes. This is a very important step.

Task:

Goal: Starting from your Part 1 LoRA-adapted model, teach it to answer questions grounded in the Sherlock Holmes canon. Show, against a baseline, whether SFT actually helped, and understand why or why not.

You will:

1. Prepare an SFT dataset (provided starter + your augmentation)
2. Train a fresh LoRA adapter on top of your Part 1 model
3. Compare your full pipeline against meaningful baselines
4. Ablate at least one design choice

1. **Environment:**

We are not enforcing any special packages, but our recommendation would be torch, transformers, peft, datasets, bitsandbytes, accelerate and trl.

- When installing keep in mind the version of the CUDA in the cluster (probably > 13) and will require newer versions of the packages – shouldn't matter if you are creating new environment.
- While we haven't taught this in class, we encourage you to read about Unsloth (<https://unsloth.ai/>) which is a python package that helps saving memory in training. It can allow you to use bigger models with the same settings. (you can complete the task without unsloth)

2. **Model choice:**

As described above, you will need to use the model from Part 1.

3. **Data:**

We provide two datasets in OpenAI messages format (JSONL, one JSON object per line)

1. Starter dataset – sft_sherlock_messages.jsonl (120 pairs)

Built by merging QA pairs from three public datasets covering 4 books – The hound of the Baskerville, The adventure of Sherlock holmes, A study in Scarlet, and the sign of four.

2. Synthetic data generated directly from the books covering 6 categories:

Category	Count
Specific_detail	180
Summarization	72
Character_theme	72
Style_imitation	36
Multi_hop	27
Unanswerable	12

The data format is a JSONL as described in class, each line is a JSON object with an OpenAI-style **messages** array

```
{"messages": [
  {"role": "system", "content": "You are an expert on the Sherlock Holmes canon by Arthur Conan Doyle. Answer questions based on: A Study in Scarlet."},
  {"role": "user", "content": "What did Holmes deduce from the man's tattoo?"},
  {"role": "assistant", "content": "Holmes deduced that the man was a retired sergeant of Marines."}
], "notes": "source=GutenQA; category=factual_recall"}
```

Remember, notes field is a metadata field and the tokenizer would ignore it!

Task: In the dataset section you will be required to generate your own dataset. At least 75 samples. The questions should be **closed questions** for easy evaluation. You can do it by any LLM synthetic data generation method you wish to, our suggestions:

- Use Ollama on your computer/cluster
- Generate with a small quantized model on the cluster
- Use a closed model
- Gemini free API for students
- Use different variations from a closed model.

Keep in mind the diversity that your generation will generate questions contextualized on the stories! Don't let an LLM generate questions without context. Remember class, don't count on the LLM's internal memory! Creativity would shine here, how to use limited context window for data generation? **Keep held-out set! (evaluation set)**

You need to provide dataset card: category, per-book distribution, design, what you kept/changed/added.

4. Training:

1. Fresh LoRA adapter on top of your Part 1 model. Freeze everything else. Do not retrain the Part 1 adapters – merge Part 1 first and then apply a fresh adapter.
2. **Chat templates** – Take a snap shot of the data and see how it is transformed with the chat template.
3. Overfitting is riskier in SFT: You would need less epochs here, learning rate is extra critical in this stage.
4. Report hyperparameters (including LoRA configs)
5. Report Training Loss as well.

5. Evaluation:

This is the important part of this work, in compared to Part 1. Evaluation in this stage is not less important than the rest of the tasks, and even more!

On your held-out test set evaluate **four** configurations:

Model	Purpose
Base model (Part 1) zero shot	Lower bound
Base model, three-shot examples	Does prompting is better than SFT?
Part 2 zero shot	Did SFT help at zero shot?
Part 3 three-shot examples	Your proposed system

Keep in mind! If the model didn't answer/didn't answer in the format – its 0! The model was trained to follow instructions, if he didn't its 0!

Report F1 and accuracy scores – Remember its closed questions so we can assess directly using F1 and accuracy scores.

6. Experiments

1. Catastrophic forgetting probe. Re-measure **OOD perplexity** on the same Wikipedia/modern-prose slice from Part 1. **Compare Part 1 model vs. Part 1 → Part 2 SFT**. How much did SFT cost you out-of-domain?
2. Data scale. Train with 25, 50, 100 pairs (randomly sampled from your training set). Plot human-rubric score vs. dataset size. Where does the curve bend? Extrapolate: how many pairs would you need for a "good" model?

7. Skip-stage. Apply Part 2 SFT directly to the base model, skipping Part 1 entirely. Is continued pre-training a prerequisite, a booster, or a distraction for SFT on this corpus?
8. LoRA rank for SFT. Try at least 2 different ranks. Does the optimal rank for SFT differ from the optimal rank you found in Part 1?
9. Response-only loss mask vs. full-sequence loss. Train one model where loss is computed only on the answer tokens, and one where loss is on the full sequence (instruction + answer). Which performs better?

You get credit for designing your own ablation and justifying it.